

Podman Essentials

Build • Run • Secure • Operate • Deploy
Containers in Production



CONTAINERS



IMAGES



NETWORKING



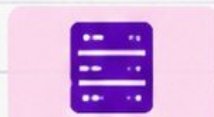
STORAGE



SECURITY



SYSTEMD



QUADLET



OBSERVABILITY



TROUBLESHOOTING



PRODUCTION



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Podman Architecture

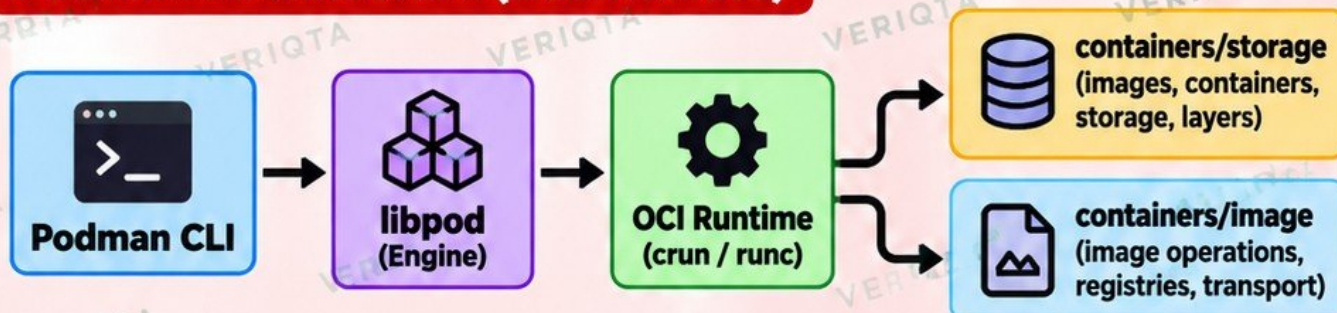
What is Podman

- Podman is a **daemonless** container engine for running OCI containers.
- It manages containers, images, networks and storage **without a background daemon**.
- Designed for security, rootless execution and integration with Linux tools such as systemd.

Key Components

- Podman CLI** – user interface
- libpod** – core engine and API
- OCI runtime** – runs containers (crun or runc)
- containers/storage** – manages images, containers and filesystem
- containers/image** – handles image operations and registries

Podman Architecture (Daemonless)



Rootless Execution

- Runs containers as a **normal user** using user namespaces.
- Does not require root** or a daemon.
- Uses **/etc/subuid** and **/etc/subgid** for UID/GID mapping.
- Provides strong isolation and better security.

How It Differs from Docker

Aspect	Podman	Docker
Architecture	Daemonless	Daemon based
Runs as root	Not required (rootless)	Usually requires root
API	Via libpod	Docker daemon API
Security	Better isolation	Larger attack surface
Integrations	systemd, Quadlet	External tools needed
Kubernetes	Generates YAML	Needs extra tooling

Production Use Cases

- Secure container runtime on Linux hosts
- Rootless containers for multi-user environments
- Systemd and Quadlet for managed services
- Works well for standalone containers and pods

Key Takeaway

- Podman runs containers without a daemon, uses standard OCI components, supports rootless execution, and integrates deeply with Linux for secure and production-ready container operations.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

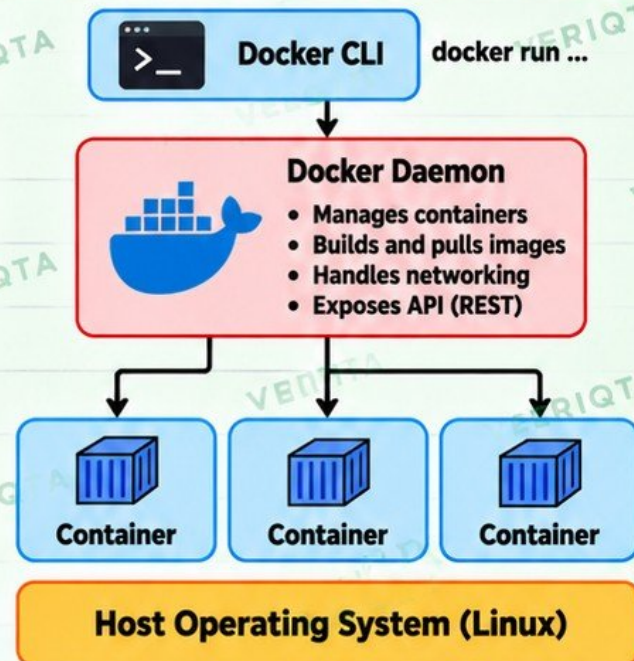


@veriqta

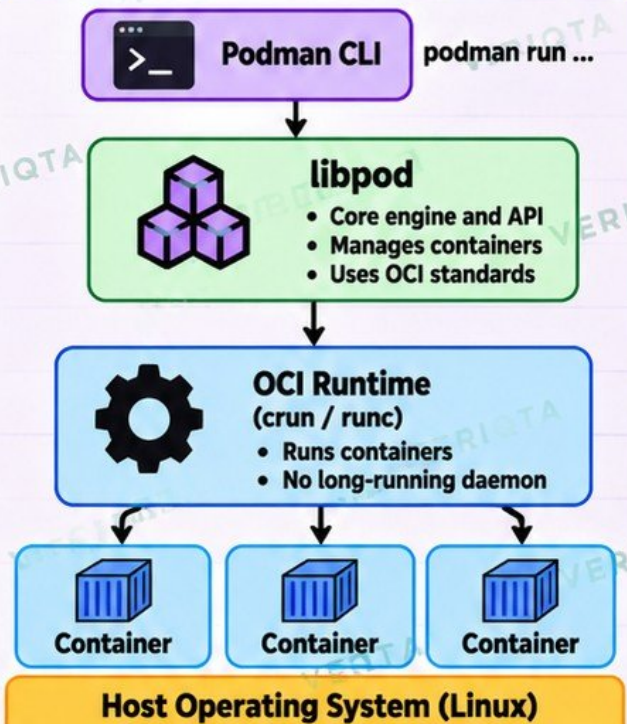


Podman vs Docker Architecture

Docker Architecture (Daemon Based)



Podman Architecture (Daemonless)



Key Differences

Aspect	Docker	Podman
Architecture	Daemon based (dockerd)	Daemonless (CLI → libpod → OCI runtime)
Runs as root	Usually requires root	Not required (rootless by default)
API	REST API exposed by daemon	Uses libpod (no daemon)
Privilege boundary	Single daemon with high privileges	No central daemon (smaller attack surface)
Failure domain	Daemon failure affects all containers	No daemon (containers isolated)
Security	Larger attack surface	Better isolation and least privilege
Systemd integration	Requires extra setup	Native support (systemd, Quadlet)
Kubernetes compatibility	Works with Docker (dockershim/CRI)	Works with OCI / CRI (no daemon needed)
Use cases	Widely used, mature ecosystem	Secure, rootless, modern Linux environments

Operational Consequences

- Docker daemon is a single point of failure.
- Root access to daemon can control all containers.
- Podman has no daemon, reducing blast radius.
- Podman fits well on multi-user systems.
- Docker is common in many existing environments.

When Architecture Matters

- Security and least privilege are important.
- Running containers on shared or multi-user hosts.
- Integrating with systemd for long-running services.
- Reducing attack surface.
- Environments where a daemon is not desirable.
- You need Docker API compatibility (Docker is simpler).

Key Takeaway

- Both run OCI containers.
- Docker uses a central daemon.
- Podman is daemonless and rootless friendly.
- Choose based on security, operational needs and environment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

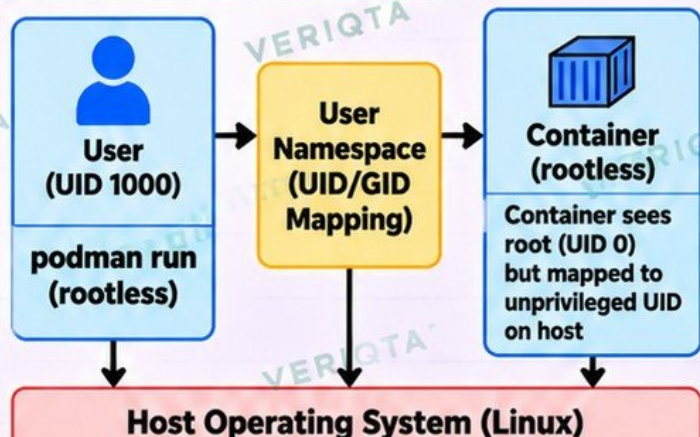


Rootless Containers

What are Rootless Containers?

- Run containers as a **normal user** without root or a daemon.
- Use Linux user namespaces for isolation.
- Each container gets its own UID/GID mapping.
- Provides strong security and reduces the blast radius.

How Rootless Works



UID/GID Mapping

- Maps container UID 0 to an unprivileged UID range on the host.
- Configured using `/etc/subuid` and `/etc/subgid`.
- Allows containers to act as root inside but without real root on the host.

/etc/subuid and /etc/subgid Example

/etc/subuid

```
user1:100000:65536
user2:165536:65536
devops:231072:65536
```

/etc/subgid

```
user1:100000:65536
user2:165536:65536
devops:231072:65536
```

Format: <username>:<start_uid>:<range>

Privilege Boundaries

Root on Host
(Real root)
Full access



Rootless
Container
(UID 0 inside)

Mapped to
Unprivileged UID
on Host
(e.g. 100000)



Security Benefits

- No root privileges required.
- Smaller attack surface.
- Container cannot directly access host root resources.
- Works well in multi-user environments.
- Better isolation using user namespaces.

Common Limitations

- Cannot bind to privileged ports (< 1024).
- Some host network and device access is restricted.
- Limited support for certain storage drivers.
- May require additional setup for systemd/Quadlet.
- Certain operations like mounting FUSE or NFS may need extra configuration.

Common Production Failures

- Incorrect `/etc/subuid` or `/etc/subgid` setup.
- Permission issues with bind mounts.
- IPv6 or port binding issues.
- Applications expecting real root privileges.
- Volume ownership mismatches.
- Confusion with file permissions on shared directories.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

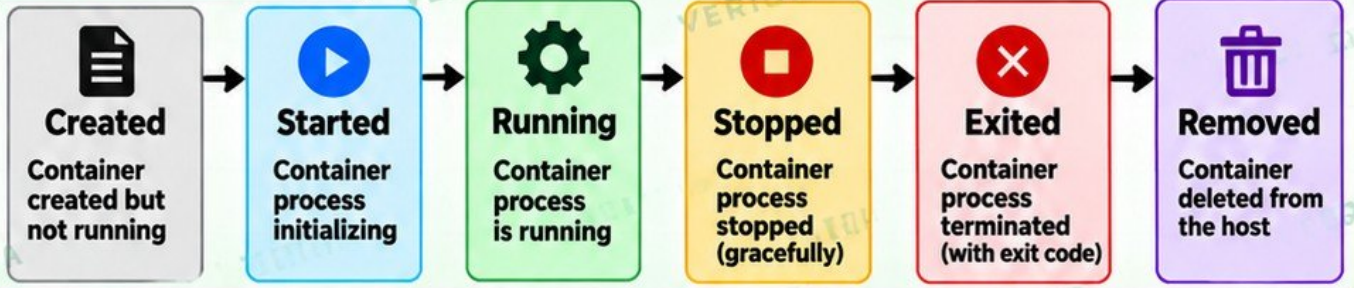


@veriqta



Container Lifecycle

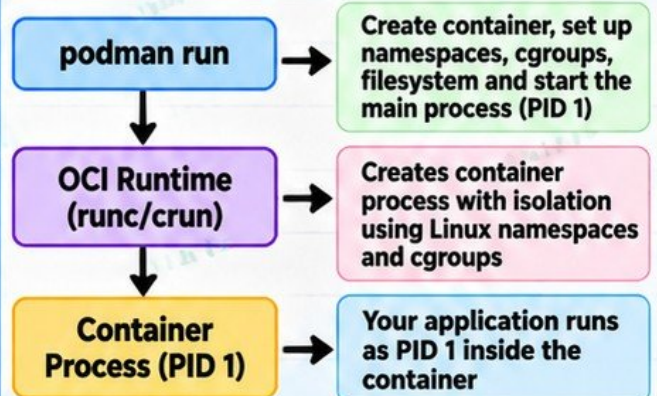
Container States



Key Commands

<code>podman create <image></code>	Create a container
<code>podman start <name></code>	Start a container
<code>podman ps</code>	List running containers
<code>podman ps -a</code>	List all containers
<code>podman stop <name></code>	Stop a container (graceful)
<code>podman kill <name></code>	Force stop (SIGKILL)
<code>podman rm <name></code>	Remove a container
<code>podman inspect <name></code>	View detailed information
<code>podman logs <name></code>	View container logs

What Happens Under the Hood?



Process Ownership and PID 1

- The main process in a container runs **as PID 1**.
- PID 1 has special signal handling **behavior**.
- If PID 1 does not **forward signals**, containers may not stop gracefully.
- Use a **proper init process** (or enable **--init**) for better signal handling and zombie reaping.

Signals and Stop Behavior

- `podman stop` sends **SIGTERM** first (default 10s).
- If the process does not exit, **SIGKILL** is sent.
- Applications should handle **SIGTERM** for graceful shutdown.
- Use **--stop-timeout** to increase the timeout.
- Use `podman kill` to send specific signals.

Exit Codes

- **0** → Successful exit
- **1-125** → Application or runtime error
- **126** → Command invoked cannot execute
- **127** → Command not found
- **137** → Killed (often due to OOMKilled)

Check with: `podman inspect <name> | grep ExitCode`

Restart Behavior

- Control restart with **--restart policy**.
- Common policies: **no** | **always** | **on-failure** | **unless-stopped**
- Helps keep containers running after failures or host reboot.
- Use with systemd or Quadlet for production services.
- Always combine with proper logging and health checks.

When a Container Dies

- The container process exits (with an exit code).
- Resources (namespaces, cgroups, mounts) are cleaned up.
- The exited container remains until removed.
- Logs remain available using `podman logs <name>`.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

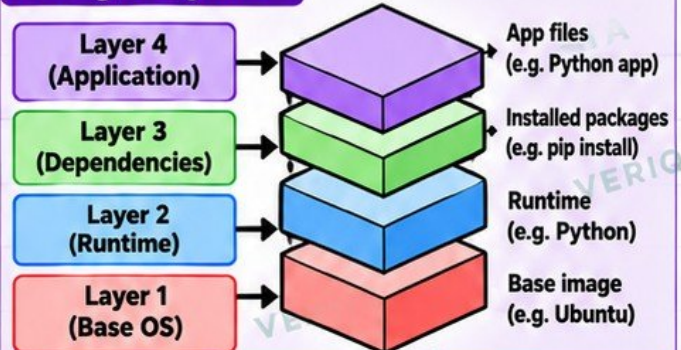


Images, Layers & Storage

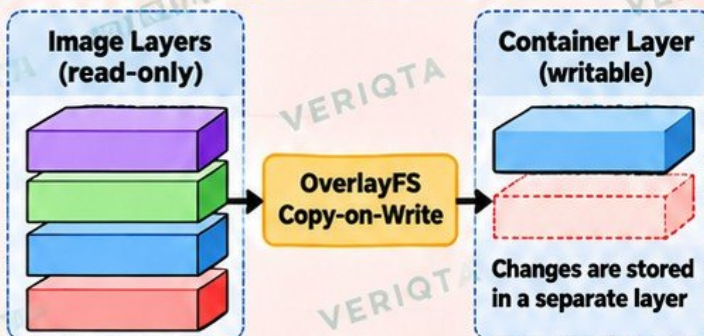
What are OCI Images?

- OCI images are **read-only** templates used to create containers.
- Built in layers, where each layer contains a set of filesystem changes.
- Stored in registries and pulled by Podman using **containers/image**.
- Follow the Open Container Initiative (OCI) standard.

Image Layers



Copy-on-Write (OverlayFS)



Original image layers stay unchanged.

Storage Drivers in Podman

Driver	Use Case / Notes
overlay (default)	Most common on Linux. Uses OverlayFS.
btrfs	Advanced features (snapshots, quotas).
zfs	Supports large storage and snapshots.
vfs	Simple but uses more disk space.

Layer Reuse and Disk Consumption

- Layers are **reused** between images to save space.
- Only new or changed layers are downloaded.
- Multiple images can share the same base layers.
- Containers add a writable layer on top.
- Unused images and containers can consume significant disk space.

Inspecting Images

<code>podman images</code>	List local images
<code>podman image inspect <image></code>	Detailed image metadata
<code>podman history <image></code>	Show image layers
<code>podman image tree <image></code>	Visualize layer hierarchy
<code>podman system df</code>	Disk usage by images, containers, volumes

Common Debugging Commands

<code>podman image inspect <image></code>	Check image config and layers
<code>podman history <image></code>	See layer size and commands
<code>podman system df -v</code>	Detailed disk usage
<code>podman image tree <image></code>	Show layer dependencies
<code>podman image prune -a</code>	Remove unused images

Common Image Issues

- Large image size due to unnecessary files.
- Duplicate layers from inefficient builds.
- Unused images consuming disk space.
- Build cache not used effectively.
- Permission or registry issues when pulling images.

Key Takeaway

- OCI images are layered and read-only.
- Podman uses storage drivers like OverlayFS for Copy-on-Write.
- Layers are shared to save space.
- Use inspection and pruning commands to manage disk usage.
- Understand layer behavior to debug image and storage issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Building Images with Podman

What is podman build?

- Creates a **container image** from a Containerfile and build context.
- Uses standard OCI image format.
- Supports advanced features like multi-stage builds, caching and build secrets.
- Produces images that can run with Podman, Docker or any OCI compliant runtime.

Basic Build Command

```
podman build -t myapp:latest .
```



Containerfile Example

```

1 FROM ubuntu:22.04
2 WORKDIR /app
3 COPY . .
4 RUN apt-get update && apt-get install -y \
5     python3 python3-pip
6 RUN pip3 install -r requirements.txt
7 CMD ["python3", "app.py"]
  
```

Containerfile

Build Context

- The **build** context is the directory you specify (e.g. `.`).
- All files in this directory are sent to Podman.
- Use a **.podmanignore** (similar to **.dockerignore**) to exclude unnecessary files.
- Keep the context small to make builds faster and more secure.

Caching

- Podman caches each layer based on the instruction and file changes.
- Unchanged layers are reused to speed up builds.
- Order instructions from least to most frequently changed.
- Use **--no-cache** to force a clean build when needed.

Multi-Stage Builds

```

1 FROM node:20 AS build
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 COPY . .
6 RUN npm run build
7
8 FROM nginx:alpine
9 COPY --from=build /app/dist /usr/share/nginx/html
  
```

Containerfile

Build Secrets

```
podman build --secret id=mytoken,src=token.txt .
```

Secrets are available during build but are not stored in the image layers. Use this for credentials, API keys, or private dependencies.

Controlling Image Size

- Use minimal base images (e.g. alpine, distroless).
- Remove unnecessary files in the same layer.
- Combine RUN commands.
- Use multi-stage builds.
- Clear package manager caches.

Reproducibility

- Pin base image versions (use tags or digests).
- Keep the Containerfile in version control.
- Use deterministic build steps.
- Avoid relying on external resources that can change.

Supply Chain Considerations

- Use trusted base images from official sources.
- Verify image signatures when available.
- Scan images for vulnerabilities.
- Keep dependencies up to date.
- Use minimal images and remove unused packages.

Key Takeaway

- **podman build** uses a Containerfile and build context to create efficient, reproducible and secure OCI images with support for caching, multi-stage builds and build secrets.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

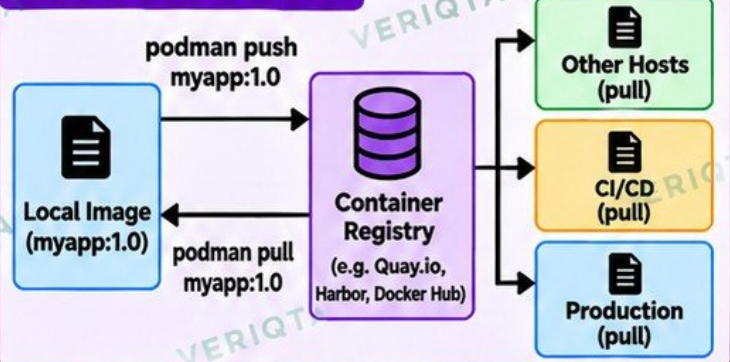


Registries & Image Trust

What are Container Registries?

- A registry stores and distributes container images.
- You can pull images from a registry and push your own images.
- Public registries are open to everyone (e.g. Docker Hub, Quay.io).
- Private registries are used inside organizations for security and control.

Pull and Push Flow



Basic Registry Commands

```
# Pull an image
podman pull nginx:latest

# Push an image
podman push myregistry.com/myapp:1.0

# Login to a registry
podman login myregistry.com

# Logout
podman logout myregistry.com
```

Registry Configuration

- Configure registries in `/etc/containers/registries.conf`.
- You can define secure (TLS) or insecure registries.
- Specify search registries and block untrusted sources.

```
# /etc/containers/registries.conf (example)
[registries.search]
registries = ["quay.io", "docker.io"]

[registries.insecure]
registries = ["myregistry.local"]
```

Authentication Methods

- Username and password (`podman login`).
- Token based authentication.
- Credential helpers (e.g. `pass`, `secret service`).
- Use CI/CD secrets for automated builds.

Tags vs Digests

- Tags are mutable (can change to a different image).
- Digests are immutable (always point to the same image).
- Use digests for production to ensure consistency.

Tag (mutable)
nginx:latest

Digest (immutable)
nginx@sha256:3a7f6e...

Private Registries

- Use private registries like Quay, Harbor or a self-hosted registry.
- Authenticate users and control access with RBAC.
- Store internal images for better security and compliance.
- Can be on-premises or in the cloud.

Image Signature Verification

- Verify image signatures to ensure the image is genuine.
- Use tools like `sigstore`, `cosign` or GPG.
- Check that the image has not been tampered with.

```
# Verify a signed image using cosign
cosign verify quay.io/your-org/myapp:1.0
```

Avoiding Untrusted Images

- Only pull images from trusted registries.
- Verify signatures for critical images.
- Use immutable digests in production.
- Scan images for vulnerabilities.
- Implement organization policies to block untrusted sources.

Key Takeaway

- Use trusted registries and secure authentication.
- Prefer immutable digests over tags for production.
- Configure registries properly and restrict untrusted sources.
- Verify image signatures and scan for vulnerabilities.
- Protect your supply chain by using private registries where needed.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

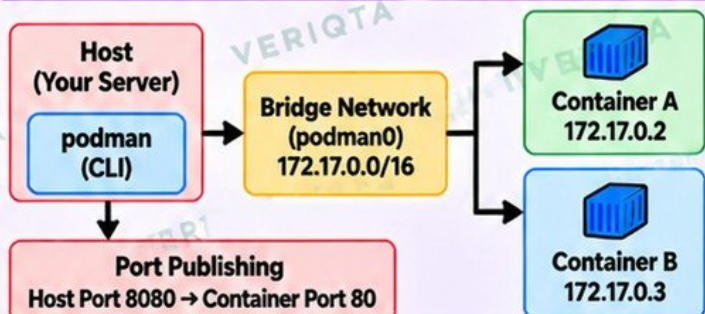


Podman Networking

What is Container Networking?

- Each container gets its own **network namespace**.
- Containers can communicate using bridges, host networking or custom networks.
- Podman provides networking without a daemon.
- Supports rootless and rootful networking.

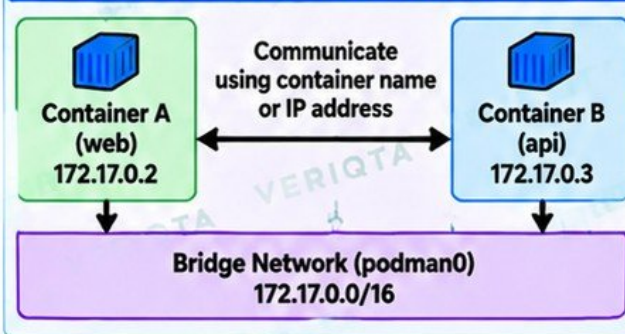
Podman Network Architecture (Bridge Mode)



Network Types in Podman

Network Type	Description	Use Case
bridge (default)	Creates a virtual bridge network (podman0).	General use (container-to-container, port publishing).
host	Uses the host network namespace.	High performance, no port mapping.
none	No network access.	Isolated containers with no networking.
custom	User-defined network with custom settings.	Isolate applications and control networking.

Container-to-Container Communication



Port Publishing

```
# Map host port 8080 to container port 80
podman run -d --name web -p 8080:80 nginx:latest
```

- Makes a container service accessible from the host.
- Host port can be a different port from the container port.

DNS in Podman

- Containers can resolve each other by container name when on the same network.
- Podman provides an internal DNS server.
- You can also use custom DNS servers.

```
podman run -d --name api --network mynet nginx:latest
podman run -d --name web --network mynet curlimages/curl
# Inside web container
curl http://api
```

Rootless Networking

- Uses **slirp4netns** for user-space networking.
- Does not require root privileges.
- Supports outbound internet access.
- Port publishing works but may have some limitations.

```
podman run -d -p 8080:80 nginx:latest
# Works in rootless mode using slirp4netns
```

Host Networking

- Container shares the host network namespace.
- No port mapping needed.
- Useful for performance-sensitive applications.

```
podman run -d --network host nginx:latest
```

Packet Flow (Port Publishing)



Key Takeaway

- Podman uses network namespaces for isolation.
- Bridge networking is default and suitable for most use cases.
- Containers can communicate using names.
- Supports rootless, host and custom networks.
- Understand the packet flow to debug networking issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Debugging Container Networking

Common Networking Issues

- DNS failures (cannot resolve names).
- Connection refused (service not listening).
- Incorrect bindings (wrong IP or port).
- Missing published ports (host cannot reach).
- Routing problems (cannot reach other networks).
- Packet loss or high latency.
- Firewall or security policy issues.

Typical Container Networking Flow



Step by Step Troubleshooting Flow

1. Verify the container is running
`podman ps`
2. Check container IP and network
`podman inspect <name> | grep -i ipaddress`
3. Verify published ports
`podman port <name>`
4. Test connectivity from host
`curl http://localhost:8080`
5. Test inside the container
`podman exec -it <name> sh`
6. Check DNS resolution
`podman exec -it <name> cat /etc/resolv.conf`
7. Inspect network configuration
`podman network ls`
`podman network inspect <network>`
8. Check logs and system evidence
`podman logs <name>`

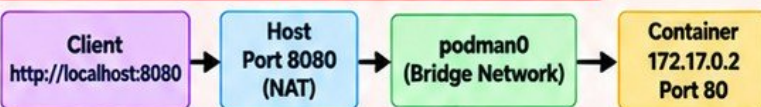
Common Errors and Fixes

Issue	Symptoms	Possible Cause	How to Investigate / Fix
DNS failure	Cannot resolve service name	DNS not configured	Check /etc/resolv.conf and network settings
Connection refused	Connection refused	Service not running or wrong port	Verify service inside container and port publishing
Incorrect bindings	Service only listening on 127.0.0.1	Bound to localhost	Bind to 0.0.0.0 or correct interface
Missing published port	Cannot reach from host	Port not published	Check podman port and -p option
Routing problems	Cannot reach other containers	Wrong network or isolation	Verify network, subnet and routes
Packet loss	Intermittent connectivity	Network issues or firewall	Use ping, traceroute, packet capture

Useful Debugging Commands

List containers	<code>podman ps -a</code>
Inspect container details	<code>podman inspect <name></code>
Check port mapping	<code>podman port <name></code>
List networks	<code>podman network ls</code>
Inspect network	<code>podman network inspect <network></code>
Test DNS inside container	<code>podman exec -it <name> cat /etc/resolv.conf</code>
Test connectivity	<code>podman exec -it <name> curl http://<host>:<port></code>
View container logs	<code>podman logs <name></code>

Packet Flow Example (Host to Container)



Key Takeaway

- Follow a systematic troubleshooting flow.
- Verify each layer: container, network, DNS, ports, routing.
- Use the right Podman commands to collect evidence.
- Most networking issues are configuration or binding related.
- Understand the packet flow to identify where it fails.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

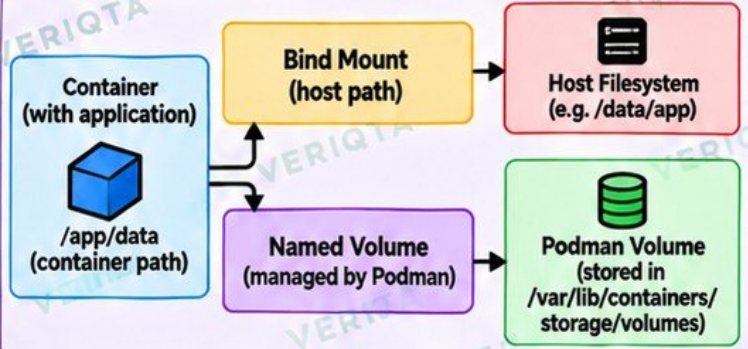


Volumes & Persistent Storage

Why Use Volumes?

- Containers are **ephemeral**, data inside is lost when a container is removed.
- Volumes and bind mounts provide **persistent storage**.
- Used for databases, application data, logs and configuration files.
- Helps with data sharing, backup and production readiness.

How Container Storage Works



Bind Mounts vs Named Volumes

Feature	Bind Mounts	Named Volumes
Location	Host filesystem path	Managed by Podman
Lifecycle	Exists outside container	Managed by Podman
Data persistence	Persists on host	Persists by default
Portability	Tied to host path	More portable
Backup	Manual (host tools)	Easier to manage
Use case	Config files, source code, logs	Application and database data
Example	<code>-v /host/path:/data</code>	<code>-v myvolume:/data</code>

Permission and Ownership

- Files created in a container may have different **UID/GID** on the host.
- In rootless mode, files are owned by your user on the host.
- You can set ownership using `--user`, `chown` or match UID/GID with the container user.
- For shared access, ensure correct permissions to avoid **permission denied** errors.

```
# Run container with specific user
podman run -d --user 1000:1000 \
-v /data:/app/data myapp:latest
```

SELinux Labeling (Linux)

- On SELinux systems, use `:Z` or `:z` to set correct labels.
- `:Z` = exclusive label (only this container can access).
- `:z` = shared label (multiple containers can access).
- Without correct labeling, you may get **permission denied** errors even with correct Unix permissions.

```
# Bind mount with SELinux label
podman run -v /data:/app/data:Z myapp:latest
```

Storage Failure Modes

- Host disk full (no space left on device).
- Permission denied (wrong UID/GID or SELinux).
- Incorrect mount path.
- Read-only filesystem.
- Host path does not exist.
- Corrupted data (unexpected container exit).
- NFS or network storage issues (if using remote storage).

Backup Considerations

- For bind mounts, use standard host backup tools.
- For named volumes, back up the volume directory (`/var/lib/containers/storage/volumes/`).
- Consider database-level backups for consistent data.
- Test restore procedures regularly.

Key Takeaway

- Use volumes for persistent data, not container filesystem.
- Choose bind mounts for host integration and config files.
- Use named volumes for application and database data.
- Ensure correct permissions and SELinux labels.
- Plan for backup and monitor disk usage.
- Understand common storage failure modes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

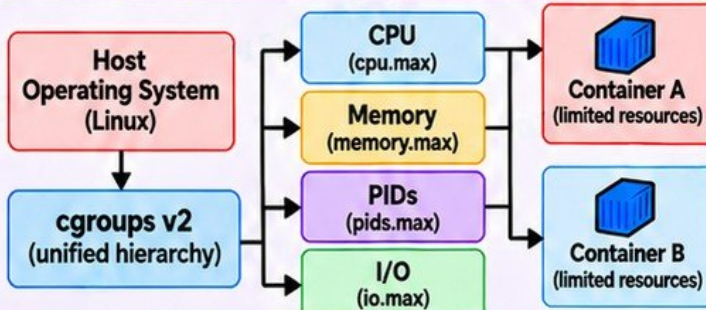


Resource Management with **cgroups**

What are cgroups?

- Control groups (**cgroups**) limit, account and isolate resource usage for processes.
- Used by Podman and the Linux kernel to manage container resources.
- Prevent containers from consuming all host resources.
- cgroups v2** is the current standard and provides a unified hierarchy.

How cgroups Work (cgroups v2)



Common Resource Controls

Resource	cgroups v2 Controller	What it Controls
CPU	cpu.max, cpu.weight	CPU usage and throttling
Memory	memory.max, memory.high	Memory limit and pressure
PIDs	pids.max	Maximum number of processes
I/O	io.max, io.weight	Disk I/O limits and priority

Setting Resource Limits (Podman)

```
# Limit CPU to 1 core
podman run -d --cpus=1 myapp:latest
# Limit memory to 512MB
podman run -d --memory=512m myapp:latest
# Limit number of processes to 100
podman run -d --pids-limit=100 myapp:latest
# Combine multiple limits
podman run -d --cpus=1 --memory=512m \
  --pids-limit=100 myapp:latest
```

CPU Limits and Throttling

- Use **--cpus** to limit CPU usage.
- When a container exceeds its limit, the kernel throttles the process (slower execution).
- Check throttling with **cpu.stat**.

```
# View CPU statistics (inside cgroup v2)
cat /sys/fs/cgroup/<container-id>/cpu.stat
```

Memory Limits and OOM Behavior

- Use **--memory** to set a memory limit.
- When a container exceeds its limit, the kernel can kill the process (OOM kill).
- Check memory usage and OOM events in **memory.stat**.

```
# View memory statistics
cat /sys/fs/cgroup/<container-id>/memory.stat
# Check kernel logs for OOM kills
dmesg | grep -i oom
```

PIDs Limit

- Use **--pids-limit** to restrict the number of processes.
- Helps prevent fork bombs and resource exhaustion.
- When the limit is reached, new processes fail to start.

```
podman run -d --pids-limit=50 myapp:latest
# Verify
podman inspect <container-id> | grep -i pids
```

Identifying Resource Pressure

- High CPU usage → check **cpu.stat** and **top/htop**.
- Memory pressure → check **memory.stat**, **dmesg** for OOM.
- Too many processes → check **pids.current** and **pids.max**.
- Disk I/O bottlenecks → check **io.stat**.
- Look for throttling, OOM kills and failed process creation as key indicators.

Noisy Neighbor Problem

- One container can consume excessive resources and impact others on the same host.
- Use cgroups to set appropriate limits.
- Monitor resource usage and set requests and limits based on actual workloads.

Key Takeaway

- cgroups v2** provides resource isolation for containers.
- Set CPU, memory, PIDs and I/O limits to prevent abuse.
- Understand throttling and OOM behavior.
- Monitor resource usage and check cgroup statistics.
- Use limits to avoid noisy neighbor problems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Podman Security Model

Key Security Principles

- Podman follows a security-by-default approach.
- Supports rootless containers for stronger isolation.
- Uses Linux kernel features like namespaces, cgroups, capabilities, seccomp and SELinux.
- Reduces the risk of container escape and privilege escalation.

Rootless vs Rootful Containers



Rootless Containers (recommended)

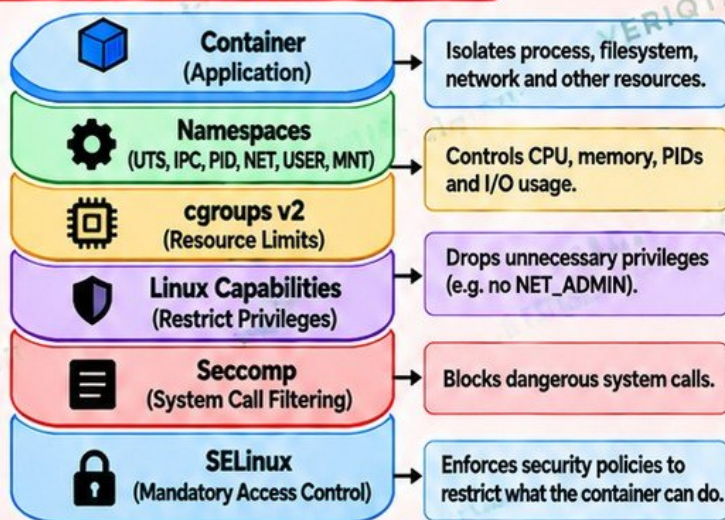
- Runs as a normal user.
- Uses user namespaces.
- No root privileges on host.
- Limited access to host resources.
- Stronger isolation and smaller blast radius.



Rootful Containers

- Runs with root privileges.
- More host access.
- Higher risk if container is compromised.
- Should be avoided unless absolutely required.

Container Isolation Layers



Linux Capabilities

- Containers do not need all root privileges.
- Use specific capabilities instead of full root.
- Common capabilities: NET_BIND_SERVICE, NET_ADMIN, SYS_TIME.
- Avoid adding unnecessary capabilities.

```
# Add a specific capability
podman run --cap-add=NET_BIND_SERVICE nginx

# Drop all capabilities except required ones
podman run --cap-drop=ALL --cap-add=NET_ADMIN nginx
```

Seccomp (System Call Filtering)

- Restricts which system calls a container can use.
- Blocks dangerous system calls (e.g. mount, ptrace).
- Podman uses a default seccomp profile.
- You can use custom profiles for stricter control.

```
# Run with the default seccomp profile
podman run --security-opt seccomp=default nginx
```

SELinux in Podman

- SELinux provides mandatory access control (MAC).
- Prevents containers from accessing unauthorized files and resources.
- Use **:Z** for private content or **:z** for shared content in bind mounts.

```
# Private content (exclusive access)
podman run -v /data:/app:Z nginx

# Shared content (multiple containers)
podman run -v /data:/app:z nginx
```

Read-Only Filesystem

- Prevents container from writing to the root filesystem.
- Reduces the impact of compromise.
- Use writable volumes only when needed.

```
podman run --read-only \
-v /data:/app:Z nginx
```

Secrets Management

- Do not store secrets in images.
- Use Podman secrets or external secret stores.
- Limit access to secrets.
- Remove secrets after use.

```
# Use a secret
podman secret create db_password ./pass.txt
podman run --secret db_password nginx
```

Why --privileged Increases Blast Radius

- Gives the container almost all host privileges.
- Disables many security restrictions.
- Allows access to host devices and kernel features.
- If the container is compromised, the attacker can do much more.
- Should only be used for specific trusted use cases.

Key Takeaway

- Use rootless containers whenever possible.
- Apply the principle of least privilege.
- Understand and use capabilities, seccomp and SELinux.
- Use read-only filesystems and proper secret management.
- Avoid **--privileged** to reduce the blast radius.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

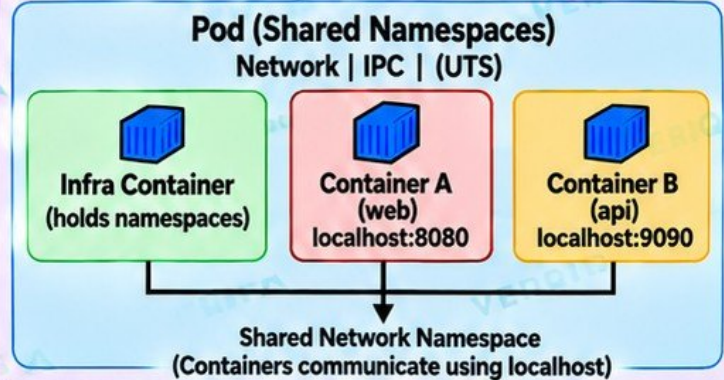


Podman Pods

What are Podman Pods?

- A pod is a group of containers that share the **same namespaces**.
- Containers in a pod share network, IPC and optionally UTS namespaces.
- Uses an **infra container** to hold the shared namespaces.
- Helps run related containers as a single logical unit.

Pod Architecture



Namespaces Shared in a Pod

Namespace	Shared?	What it Means
Network	Yes	All containers share the same network (same IP).
IPC	Yes	Containers can communicate using shared IPC (e.g. sockets).
UTS (optional)	Yes (if enabled)	Same hostname for containers in the pod.
PID	No (default)	Each container has its own process namespace (isolated).
Mount	No (default)	Each container has its own filesystem (unless explicitly shared).

Example Pod Setup

```

# Create a pod
podman pod create --name webapp

# Run a web container in the pod
podman run -d --pod webapp --name web nginx

# Run an API container in the same pod
podman run -d --pod webapp --name api myapi:latest

# List pod and containers
podman pod ps
podman ps --pod
  
```

How Containers Communicate in a Pod

- Containers in the same pod share the network namespace.
- Use localhost to communicate between containers.
- No need to publish ports between containers in the same pod.
- You only publish ports at the pod level for external access.



Pod vs Separate Containers

Feature	Pod (Same Pod)	Separate Containers
Network	Shared	Isolated (needs port mapping or custom network)
IPC	Shared	Isolated
Hostname	Shared (optional)	Separate
Lifecycle	Managed together	Managed independently
Use case	Tightly coupled services	Loosely coupled services
Example	Web + sidecar, app + log collector	Independent applications

When to Use Pods?

- ✓ Running tightly coupled containers (e.g. main app + sidecar).
- ✓ Containers need to share the same network (use localhost).
- ✓ Simpler management of related containers.
- ✓ Useful for patterns like app + log collector, app + proxy.

When Separate Containers are Cleaner?

- ✗ Services are independent.
- ✗ Need different networks or port mappings.
- ✗ Different lifecycle requirements.
- ✗ Better isolation and security.
- ✗ Easier to scale individually.

Key Takeaway

- Pods share namespaces (especially network).
- Uses an infra container.
- Simplifies running related containers.
- Use pods for tightly coupled components.
- Use separate containers for independent services.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

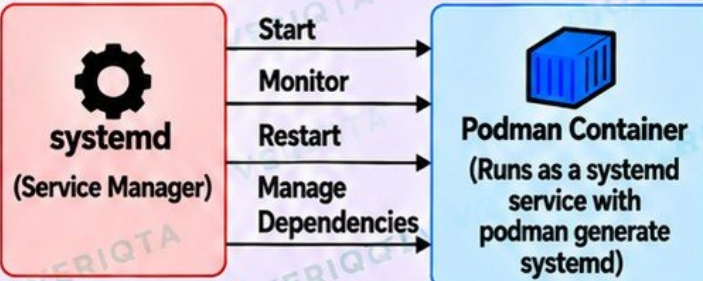


Podman + systemd

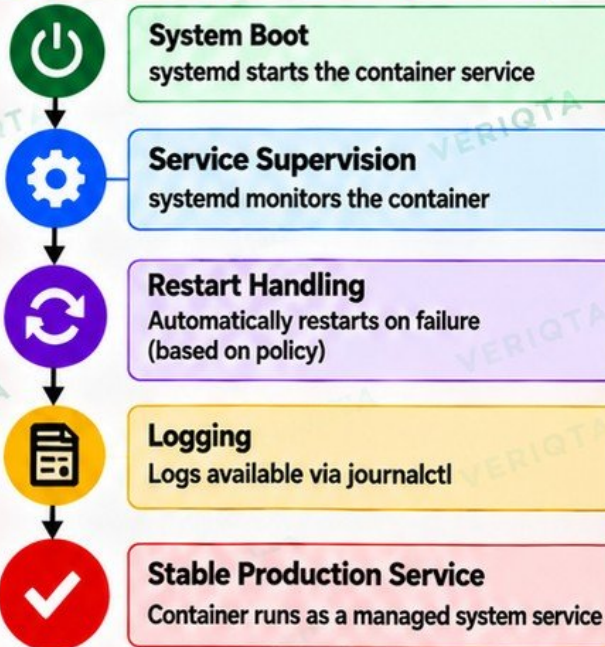
Why systemd Matters?

- Keeps containers running as reliable system **services**.
- Automatically starts containers **on boot**.
- Monitors and **restarts** containers if they fail.
- Manages **dependencies** between services.
- Provides centralized logging and operational visibility.
- Ideal for long-running workloads in production.

How It Works



Typical Lifecycle with systemd



Generate a systemd Service

```
# Create a container
podman run -d --name myapp nginx
# Generate systemd service file
podman generate systemd --name myapp --files
# Move to systemd directory
sudo mv container-myapp.service /etc/systemd/system/
# Enable and start the service
sudo systemctl enable myapp.service
sudo systemctl start myapp.service
```

Useful systemctl Commands

Task	Command
Check service status	systemctl status myapp.service
Start service	systemctl start myapp.service
Stop service	systemctl stop myapp.service
Enable at boot	systemctl enable myapp.service
Disable at boot	systemctl disable myapp.service
View logs	journalctl -u myapp.service
Restart service	systemctl restart myapp.service

Service File Example (Generated)

```
[Unit]
Description=Podman container-myapp.service
After=network-online.target
Wants=network-online.target

[Service]
ExecStart=/usr/bin/podman start myapp
ExecStop=/usr/bin/podman stop -t 10 myapp
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Key Benefits

- ✓ Automatic start on boot.
- ✓ Service supervision and auto-restart.
- ✓ Dependency management (e.g. network, storage).
- ✓ Centralized logging with journalctl.
- ✓ Better reliability for production workloads.
- ✓ Integrates with the operating system.

Key Takeaway

- Use systemd for long-running Podman containers.
- It provides automatic startup, monitoring and recovery.
- Makes containerized services behave like native system services.
- Essential for production lifecycle management.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

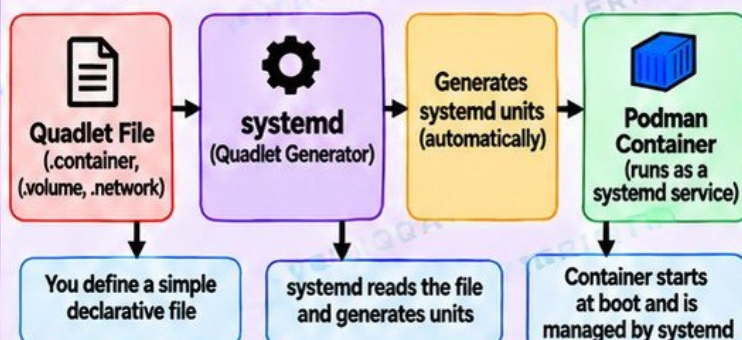


Quadlet

What is Quadlet?

- Quadlet allows you to define Podman containers, volumes, networks and more using simple **declarative files**.
- Uses **systemd** for service management.
- Automatically generates the required systemd units.
- Simplifies container management.
- Ideal for production and long-running workloads.

How Quadlet Works



Quadlet File Types

File Type	Extension	Purpose
Container	.container	Defines a Podman container and its settings.
Volume	.volume	Defines a Podman volume.
Network	.network	Defines a Podman network.
Pod	.pod	Defines a Podman pod (multiple containers).
Kube	.kube	Runs Kubernetes workloads using Podman.

Example: .container File

```

[Unit]
Description=My Nginx Container
After=network-online.target
Wants=network-online.target

[Container]
Image=nginx:latest
ContainerName=webapp
PublishPort=8080:80
Restart=always
Network=webnet

[Install]
WantedBy=multi-user.target
  
```

Example: .network File

```

[Network]
Description=Custom Podman Network
Driver=bridge
Subnet=10.88.0.0/24
Gateway=10.88.0.1
  
```

Example: .volume File

```

[Volume]
Description=App Data Volume
Driver=local
VolumeName=appdata
  
```

Example: .pod File

```

[Pod]
Description=Web Application Pod
PublishPort=8080:80
Network=webnet

[Install]
WantedBy=multi-user.target
  
```

Key Benefits

- ✓ Declarative and simple configuration.
- ✓ Automatic systemd integration.
- ✓ Handles dependencies between services.
- ✓ Starts containers at boot.
- ✓ Easier to maintain than generated units.
- ✓ Supports containers, volumes, networks and pods.
- ✓ Ideal for modern Podman deployments.

Dependency Management

- Use standard systemd directives (After, Wants, Requires).
- Manage service startup order (e.g. network before container).
- Supports dependencies between containers, volumes and networks.

Why Prefer Quadlet?

- ✓ No need to manually maintain generated unit files.
- ✓ Clean and declarative configuration.
- ✓ Better for production and large deployments.
- ✓ Integrates naturally with systemd.
- ✓ Simplifies lifecycle management.

Key Takeaway

- Use Quadlet for a clean, declarative and production-ready setup.
- Define .container, .volume, .network and related files.
- Leverage systemd for reliable startup, dependency management and lifecycle control.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

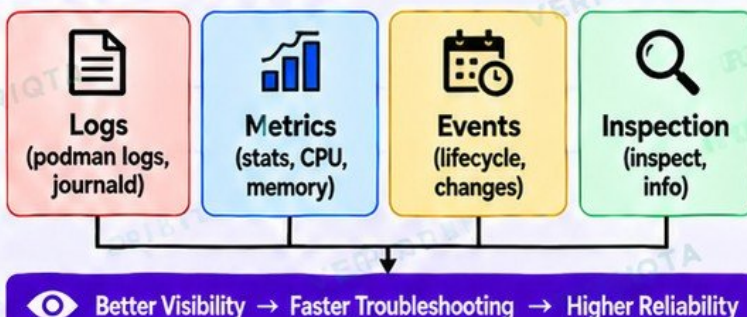


Logs & Observability

Why Logs & Observability?

- Helps you understand what is happening in your containers and applications.
- Speeds up **troubleshooting** and **incident response**.
- Provides visibility into **performance**, **resource usage** and **system health**.
- Helps you distinguish between **application** failures and **runtime** failures.

Observability Stack



Key Commands

Command	What it does
podman logs	View container logs
journalctl	View systemd/journald logs
podman stats	Live resource usage (CPU, memory, etc)
podman events	Show container events
podman inspect	Detailed container information
podman ps -a	List containers with status
podman wait	Wait for container and get exit code

Logs Examples

```

# View container logs
podman logs myapp

# Follow logs
podman logs -f myapp

# View journald logs
journalctl -u podman-<container-id> -f

# Get container stats
podman stats

# Show container events
podman events --since 1h
  
```

Health & Resource Signals

	CPU	High usage, throttling, CPU pressure
	Memory	High usage, OOM events, memory pressure
	Health	Health check status, container state
	Exit Codes	0 = success, non-zero = failure

Application vs Runtime Failures

Application Failure

- ✓ Container runs but app errors.
- ✓ Exit code from the application.
- ✓ Seen in application logs.
- ✓ Usually fixable in the code, configuration or dependencies.

Runtime Failure

- ✗ Container crashes or exits.
- ✗ Non-zero exit code (e.g. 137).
- ✗ OOM, permission or runtime errors.
- ✗ Seen in podman events, journald or container logs.
- ✗ Often related to resources, security or environment.

Troubleshooting Flow



Key Takeaways

- ✓ Use multiple signals, not just logs.
- ✓ Check journald for systemd context.
- ✓ Monitor resource usage and health.
- ✓ Use events and inspect for details.
- ✓ Know the difference between app and runtime failures.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Troubleshooting Failed Containers

Common Failure Symptoms

- ✗ Container exits immediately
- 🔄 Container keeps restarting
- ❗ Application not responding
- 🌐 Cannot access service (connection refused)
- 🕒 Container is running but unhealthy
- 🔥 High CPU or memory usage
- 💾 Storage or permission errors
- 🌐 Network/DNS connectivity issues

Key Podman Commands

Command	Purpose
podman ps -a	List all containers
podman inspect <name>	Detailed container info
podman logs <name>	View container logs
podman events	Show recent events
podman stats	Live resource usage
podman top <name>	View running processes
podman port <name>	Check published ports
podman exec -it <name> sh	Access container shell
podman restart <name>	Restart container

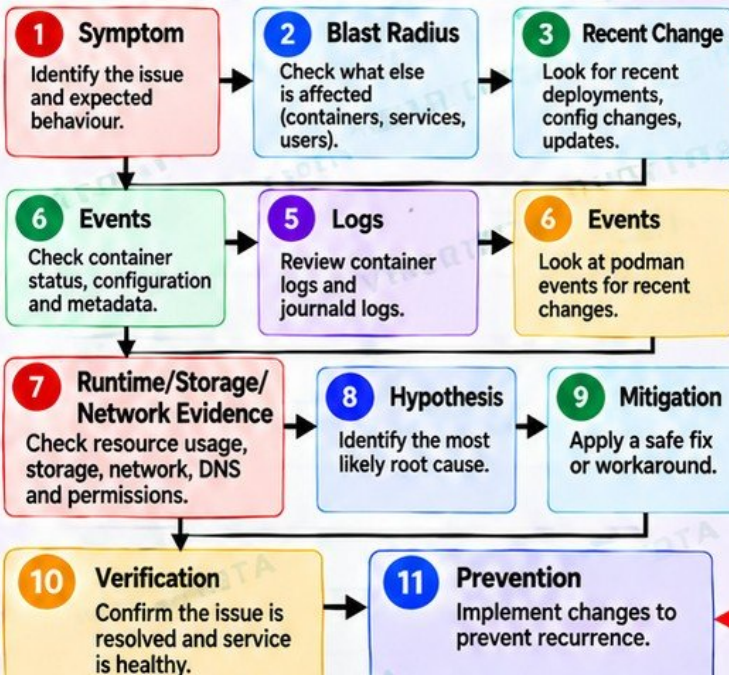
Where to Look for Evidence

📄	Container Status	podman ps -a, podman inspect
📄	Application Logs	podman logs, journald
📅	Events	podman events
📊	Resource Usage	podman stats, top, CPU/memory metrics
💾	Storage	Volume mounts, permissions, disk space
🌐	Network	Ports, DNS, connectivity, firewall
⚙️	Runtime	Exit codes, restart policy, cgroups, OOM

Exit Code Examples

Exit Code	Meaning
0	Successful exit
1	General application error
125	Podman/runtime error (container not started)
126	Command found but not executable
127	Command not found
137	Killed (often OOM - out of memory)
143	Terminated (SIGTERM)

Senior Troubleshooting Workflow



Common Root Causes

- ⚙️ Application error or misconfiguration
- 📄 Incorrect environment variables
- 💾 Missing or incorrect volume mounts
- 🌐 Network or DNS resolution issues
- 🛡️ Permission or SELinux policy problems
- 🖥️ Resource limits (CPU, memory, PIDs)
- 🔄 Crank loops due to failing health checks
- 🔧 Image or runtime compatibility issues
- ⚠️ Recent changes causing regression

Quick Checks

- ✓ Check container exit code (podman ps -a)
- ✓ View last logs (podman logs)
- ✓ Look at events (podman events)
- ✓ Verify configuration (podman inspect)
- ✓ Check resource usage (podman stats)
- ✓ Validate network connectivity (DNS, ports)
- ✓ Check volume mounts and permissions
- ✓ Review recent changes
- ✓ Test a clean container from the same image

Key Takeaways

- ✓ Follow a structured and consistent troubleshooting workflow.
- ✓ Use logs, events and inspection to find real evidence.
- ✓ Check runtime, storage and network layers, not just application logs.
- ✓ Identify the root cause and apply a safe mitigation.
- ✓ Verify the fix and implement preventive measures.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Updates, Rollbacks & Recovery

Why Updates & Rollbacks Matter?

- ✓ Keep your containers secure and up to date.
- ✓ Get new features and bug fixes.
- ✓ Reduce downtime during changes.
- ✓ Allow quick rollback when something goes wrong.
- ✓ Protect persistent data and configuration.
- ✓ Maintain reliability and service continuity.

Update Workflow (High Level)

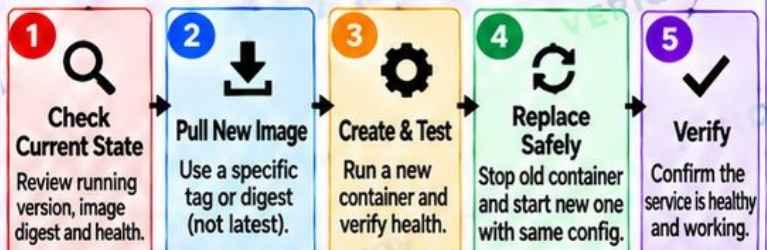


Image Tags vs Digests

Tags (e.g. latest, v1.2.0)

- ! Can change over time.
- ! Not immutable.
- ! May point to a different image later.
- ! Avoid using latest in production.

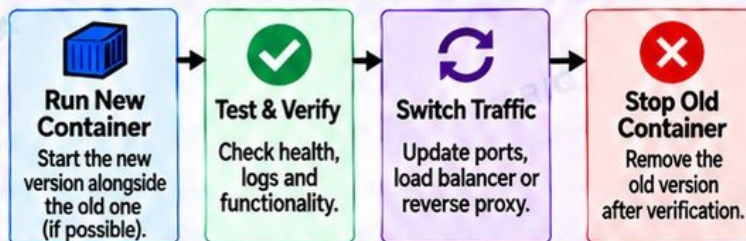
Digests (e.g. @sha256:abc...)

- ✓ Immutable (never changes).
- ✓ Always points to the same image.
- ✓ Safer for production.
- ✓ Use digests for predictable deployments.

Key Podman Commands

Command	Purpose
podman pull <image>	Pull a specific image.
podman images	List local images.
podman ps -a	List containers.
podman stop <name>	Stop running container.
podman rm <name>	Remove container.
podman run ...	Start new container with updated image.
podman inspect <name>	Check image, config and state.
podman logs <name>	View logs after update.

Controlled Replacement (Zero/Low Downtime)



Rollback Strategies

- 🔄 Keep the previous image tag or digest.
- 📦 Use immutable images for consistent rollbacks.
- 📄 Recreate the old container with the previous version.
- ⚙️ Keep configuration and persistent data unchanged.
- 🕒 Test the rolled back version and verify health.

Example: Update with a Specific Tag

```

# Pull the new image (specific tag)
podman pull myapp:v1.2.0
# Stop the current container
podman stop myapp
# Remove the old container
podman rm myapp
# Start the updated container
podman run -d --name myapp -p 8080:80 myapp:v1.2.0
  
```

Example: Rollback to Previous Version

```

# Pull the previous image
podman pull myapp:v1.1.0
# Stop the current (broken) container
podman stop myapp
# Remove the current container
podman rm myapp
# Start the previous working version
podman run -d --name myapp -p 8080:80 myapp:v1.1.0
  
```

Important Considerations

- 📦 Protect persistent data (volumes, databases, configs).
- 📄 Do not assume in-place updates are safe.
- ⚙️ Check application compatibility and environment variables.
- ⚠️ Verify health, logs and metrics after every update.
- ⚙️ Plan for rollback before making changes.

Key Takeaways

- ✓ Use specific tags or digests, not latest.
- ✓ Test updates before replacing the current container.
- ✓ Always have a rollback plan.
- ✓ Protect persistent state and configuration.
- ✓ Verify service health after updates.
- ✓ Avoid unsafe in-place assumptions.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Production Podman Architecture

What is Production Architecture?

- ✓ A reliable, secure and maintainable setup for real workloads.
- ✓ Uses best practices for containers, networks, storage, security and monitoring.
- ✓ Designed for stability, scalability and easy operations.
- ✓ Reduces risks and makes recovery easier.

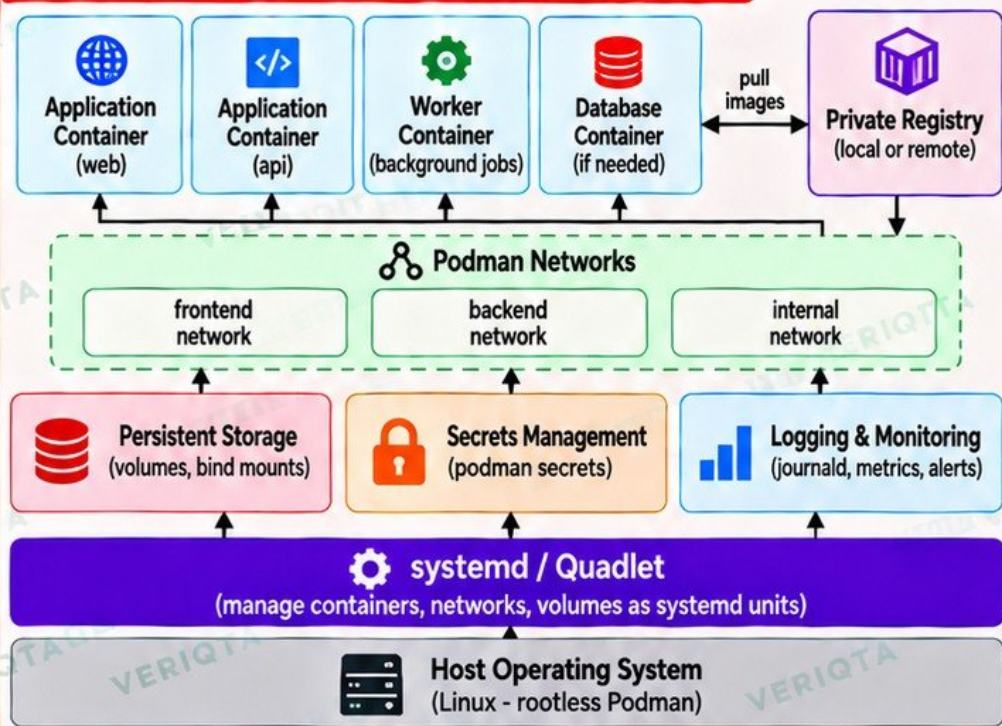
Key Design Principles

- ✓ Security by default (rootless, least privilege).
- ✓ Automation with systemd/Quadlet.
- ✓ Persistent and reliable storage.
- ✓ Proper network design and isolation.
- ✓ Observability and monitoring.
- ✓ Plan for failures and recovery.

Core Components

- Rootless Workloads
- Systemd / Quadlet
- Container Registry
- Networks
- Persistent Storage
- Secrets Management
- Logging
- Monitoring
- Resource Boundaries
- Backups & Recovery
- Host Failure Handling

Production Podman Host Architecture



Security Considerations

- ✓ Use rootless containers.
- ✓ Apply least privilege and Linux capabilities.
- ✓ Use seccomp and SELinux.
- ✓ Keep images minimal and trusted.
- ✓ Store sensitive data using podman secrets.
- ✓ Isolate networks and limit exposure.
- ✓ Regularly update images and the host.

Resource Boundaries

- CPU limits and reservation.
- Memory limits.
- PID limits.
- Prevent noisy neighbor issues.
- Use cgroups v2 for better control.

Backup Strategy

- ✓ Back up persistent volumes.
- ✓ Back up configuration files (Quadlet units, systemd).
- ✓ Document recovery steps.
- ✓ Test restore periodically.
- ✓ Consider off-site backups for critical data.

Host Failure Considerations

- Plan for host hardware failure.
- Keep backups of data and configs.
- Use infrastructure redundancy (if possible).
- Document rebuild steps.
- Design for fast recovery and minimal downtime.

Key Takeaways

- ✓ Use rootless Podman for security.
- ✓ Leverage systemd/Quadlet for reliable management.
- ✓ Design networks, storage and secrets properly.
- ✓ Enable logging and monitoring.
- ✓ Set resource boundaries.
- ✓ Have a backup and recovery plan.
- ✓ Always consider host failure scenarios.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Senior Engineer Decision Guide

What This Guide Helps You With?

- ✓ Understand the differences clearly.
- ✓ Choose the right platform for your use case.
- ✓ Know the operational trade-offs.
- ✓ Avoid common mistakes.
- ✓ Make better decisions in real projects.

When to Consider Each Platform?



Podman

- Single host workloads
- Rootless containers
- Secure by default
- Systemd integration
- Simpler operations



Docker

- Simple to use
- Developer friendly
- Single host workloads
- Wide ecosystem
- Great for local dev and small deployments



Kubernetes

- Multi host orchestration
- Large scale deployments
- Advanced self healing
- Complex environments
- Enterprise workloads

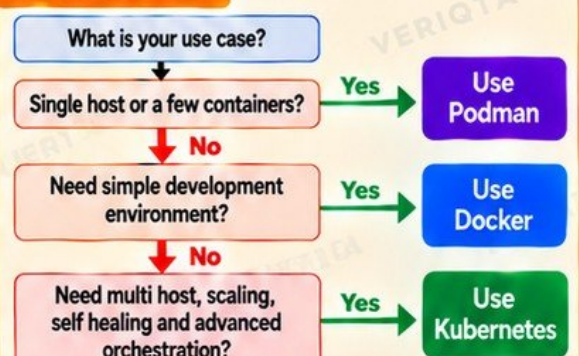
Detailed Comparison

Aspect	Podman	Docker	Kubernetes
Operational Complexity	Low	Low to Medium	High
Primary Use Case	Single host/Container host	Single host	Multi host orchestration
Orchestration	No (use pods/systemd)	No (standalone)	Yes (built in)
Security Model	Rootless, least privilege	Rootful by default (rootless optional)	Strong (RBAC, namespaces, policies)
Scaling	Limited (single host)	Limited (single host)	High (multi node auto scaling)
Failure Domain	Host level	Host level	Cluster level (spreads across nodes)
System Integration	Excellent (systemd/Quadlet)	Basic (systemd possible)	Advanced (controllers, operators)
Best For	Servers, edge, secure hosts	Development, simple apps	Production at scale
Resource Overhead	Low	Low	High

Strengths and Limitations

Podman	Docker	Kubernetes
✓ Rootless and secure ✓ No daemon required ✓ Works well with systemd ✓ Great for single host use cases	✓ Easy to use ✓ Large community ✓ Great for development ✓ Wide tooling support	✓ Built for large scale ✓ Self healing and resilient ✓ Rich ecosystem ✓ Handles complex workloads
✗ No built in orchestration ✗ Limited scaling ✗ Not designed for multi host	✗ Rootful by default ✗ Limited orchestration ✗ Not ideal for large scale production	✗ High operational complexity ✗ Steeper learning curve ✗ Overkill for simple use cases ✗ Requires multiple nodes

Decision Flow



Key Considerations

- Operational complexity and team expertise.
- Security and compliance requirements.
- Current and future scaling needs.
- Data persistence and failure tolerance.
- Single host vs multi host requirements.
- Cost and resource overhead.
- Integration with existing infrastructure.
- Long term maintenance and support.

Key Takeaways

- ✓ Podman is ideal for secure single host deployments.
- ✓ Docker is best for development and simple workloads.
- ✓ Kubernetes is the right choice for large scale orchestration.
- ✓ Match the platform to your real operational needs.
- ✓ Avoid using Kubernetes when Podman or Docker is enough.
- ✓ Consider future growth, security and team skills.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

